

Consumo de servicios web con REST API

Modalidad: Práctica de laboratorio, individual

1. Propósito de la actividad

Que el estudiante consume servicios web a través de una REST API, primero con **Postman** y después con **Python** (lenguaje multiparadigma), interpretando respuestas en formato **JSON** y comparándolas con **XML**, para reafirmar los temas de la unidad:

- APIs y análisis de JSON.
- REST API utilizando Postman.
- REST API utilizando un lenguaje multiparadigma.
- Servicios web con API REST, JSON o XML.

2. Materiales

- Equipo de cómputo con acceso a Internet.
- Postman (aplicación de escritorio o web).
- Python 3 y un IDE (VS Code, PyCharm u otro).
- Librería `requests` (instalar con `pip install requests`).
- APIs públicas que **no requieren registro ni llave**:
 - o JSONPlaceholder: `https://jsonplaceholder.typicode.com` (API de práctica con recursos `posts`, `users`, `comments`, etc.).
 - o Open-Meteo: `https://api.open-meteo.com/v1/forecast` (pronóstico del clima).

*Nota: JSONPlaceholder **simula** las operaciones de escritura. Responde como si el POST, PUT o DELETE hubieran funcionado, pero los datos no se guardan realmente. Es normal que un GET posterior no muestre el cambio.*

```
Símbolo del sistema
Microsoft Windows [Versión 10.0.26300.9457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\hazie>pip install requests
Collecting requests
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Downloading charset_normalizer-3.5.2-cp313-cp313-win_amd64.whl.metadata (47 kB)
Collecting idna<4,>=2.5 (from requests)
  Downloading idna-3.20-py3-none-any.whl.metadata (7.2 kB)
Collecting urllib3<3,>=1.26 (from requests)
  Downloading urllib3-2.8.0-py3-none-any.whl.metadata (7.4 kB)
Collecting certifi>=2023.5.7 (from requests)
  Downloading certifi-2026.7.22-py3-none-any.whl.metadata (2.5 kB)
Downloading requests-2.34.2-py3-none-any.whl (73 kB)
Downloading charset_normalizer-3.5.2-cp313-cp313-win_amd64.whl (206 kB)
Downloading idna-3.20-py3-none-any.whl (69 kB)
Downloading urllib3-2.8.0-py3-none-any.whl (135 kB)
Downloading certifi-2026.7.22-py3-none-any.whl (136 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
Successfully installed certifi-2026.7.22 charset_normalizer-3.5.2 idna-3.20 requests-2.34.2 urllib3-2.8.0

C:\Users\hazie>
```

3. Parte A. Repaso conceptual (20 min)

Responde en tu cuaderno o en un documento, con tus propias palabras:

- 1. **¿Qué es una API y qué significa que una API sea REST?** Una API es un conjunto de reglas que permite que dos aplicaciones se comuniquen. Una API Rest significa que sigue un estilo específico basado en la web.
- 2. **Describe la diferencia entre cliente y servidor en una petición HTTP.** Cliente es la entidad que inicia la comunicación enviando una solicitud, y el servidor es la entidad que recibe la solicitud, procesa la información y envía una respuesta.

3. Completa la tabla de métodos HTTP:

Método	Operación CRUD	Para qué se usa	Código de éxito típico
GET	Leer	Consulta y obtiene datos del servidor	200 OK
POST	Crear	Enviar datos para crear un nuevo recurso	201 Created
PUT	Actualizar	Reemplazar por completa un recurso existente	200 OK y 204 No Content
PATCH	Actualizar	Modificar parcialmente un recurso existente	200 OK y 204 No Content

Método	Operación CRUD	Para qué se usa	Código de éxito típico
DELETE	Eliminar	Eliminar un recurso específico del servidor	200 OK y 204 No Content

- 4. Explica qué significan los códigos de estado: **200, 201, 204, 400, 401, 404 y 500.**
200: Éxito. El servidor procesó la solicitud y devolvió los datos.

201: Creado. La solicitud tuvo éxito y se creó un nuevo recurso.
204: Sin contenido. Éxito, pero la respuesta no tiene cuerpo.
400: Solicitud incorrecta. El cliente envía datos con errores de sintaxis.
401: No autorizado. Se requiere autenticación válida para acceder.
404: No encontrado. El recurso solicitado no existe en el servidor.
500: Error interno. El servidor falló al procesar la solicitud.

5. Escribe un objeto JSON que representa a un dispositivo de red con: nombre, dirección IP, modelo, estado (activo o inactivo) y una lista de interfaces. Indica qué tipo de dato es cada valor.

```
{
  "nombre": "Router-Central-01",
  "direccion_ip": "192.168.10.1",
  "modelo": "Cisco ISR 4331",
  "estado": true,
  "interfaces": ["GigabitEthernet0/0/0", "GigabitEthernet0/0/1", "Vlan100"]
}
```

4. Parte B. Consumo de la API con Postman (60 min)

Crea una **colección** en Postman llamada `Unidad2_Act2_TuNombre` y dentro de ella una petición por cada ejercicio. Toma captura de pantalla de la petición y de la respuesta (cuerpo, código de estado y encabezados).

#	Petición	URL	Qué debes observar
B1	GET lista de publicaciones	<code>https://jsonplaceholder.typicode.com/posts</code>	Código 200 y que el cuerpo es una lista (arreglo) de objetos.
B2	GET un recurso	<code>https://jsonplaceholder.typicode.com/posts/1</code>	Campos <code>id</code> , <code>userId</code> , <code>title</code> , <code>body</code> .
B3	GET con parámetros	<code>https://jsonplaceholder.typicode.com/posts?userId=3</code>	Solo regresa publicaciones del usuario 3. Anota cuántas son.
B4	GET recurso anidado	<code>https://jsonplaceholder.typicode.com/posts/1/comments</code>	Comentarios de la publicación 1.
B5	POST crear	<code>https://jsonplaceholder.typicode.com/posts</code>	En <i>Body</i> → <i>raw</i> → <i>JSON</i> envía un objeto con <code>title</code> , <code>body</code> y <code>userId</code> . Código 201 y un <code>id</code> nuevo.
B6	PUT reemplazar	<code>https://jsonplaceholder.typicode.com/posts/1</code>	Envía un objeto completo. Compara con B2.

B7	PATCH modificar	https://jsonplaceholder.typicode.com/posts/1	Envía solo {"title": "Nuevo título"}.
B8	DELETE eliminar	https://jsonplaceholder.typicode.com/posts/1	Código 200 y cuerpo vacío {}.
B9	GET recurso inexistente	https://jsonplaceholder.typicode.com/posts/9999	Código 404. Explica qué significa.
B10	GET con parámetros de consulta	https://api.open-meteo.com/v1/forecast?latitude=20.59&longitude=-	Temperatura y viento actuales de Querétaro.

#	Petición	URL	Qué debes observar
		<code>100.39&current=temperature_2m,wind_speed_10m</code>	Identifica las unidades en el campo <code>current_units</code> .

B1

Unidad2_Act2_Jairo

GET B1

Unidad2_Act2_Jairo > B1

GET <https://jsonplaceholder.typicode.com/posts>

Docs

Params

Authorization

Headers 10

Body

Scripts

Settings

Query Params

Key	Value
Key	Value

Body

Cookies

Headers 22

Test Results

200 OK

303 ms

7.77 KB

Save Response

{}

JSON

Preview

Visualize

```
1  [
2    {
3      "userId": 1,
4      "id": 1,
5      "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6      "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
7    },
8    {
9      "userId": 1,
10     "id": 2,
11     "title": "qui est esse",
12     "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
13   },
14   {
15     "userId": 1,
16     "id": 3,
```

main

Terminal

Console

Standing by for launch

Globals

Vault

Tools

Invite

Upgrade

Jairo Haziel Rosas Enríquez

1225100376.jhre@gmail.com

Jairo Haziel Rosas Enríquez's Te...

Free plan

Switch to

Billing & resource usage

Public elements

Postman keys

Audit logs

Sign out

No Environment

Save

Share

Send

Cookies

Bulk Edit

B2

The screenshot shows the Postman interface with a workspace named "Jairo Haziel Rosas Enriquez's Workspace". On the left, a collection named "Unidad2_Act2_Jairo" is expanded, showing requests B1 through B8. Request B2 is selected. The main panel displays a GET request to the URL "https://jsonplaceholder.typicode.com/posts/1". The response is a 200 OK status with a response time of 252 ms and a body of 1.08 KB. The response body is shown in JSON format:

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
6 }
```

B3

The screenshot shows the Postman interface with the same workspace. The collection "Unidad2_Act2_Jairo" is expanded, and request B3 is selected. The main panel displays a GET request to the URL "https://jsonplaceholder.typicode.com/posts/userId=3". The response is a 200 OK status with a response time of 593 ms and a body of 1.86 KB. The response body is shown in JSON format:

```
1 [
2   {
3     "userId": 3,
4     "id": 21,
5     "title": "asperiores ea ipsam voluptatibus modi minima quia sint",
6     "body": "repellat aliquid praesentium dolorem quo\nsed totam minus non itaque\nnihil labore molestiae sunt dolor eveniet hic recusandae veniam\ntempora et tenetur expedita sunt"
7   },
8   {
9     "userId": 3,
10    "id": 22,
11    "title": "dolor sint quo a velit explicabo quia nam",
12    "body": "eos qui et ipsum ipsam suscipit aut\nsed omnis non odio\nexpedita earum mollitia molestiae aut atque rem suscipit\nnam impedit esse"
13  },
14  {
15    "userId": 3,
16    "id": 23,
```

B4

The screenshot shows the Postman interface with a workspace named "Jairo Haziel Rosas Enriquez's Workspace". On the left, a sidebar lists "COLLECTIONS" including "Programacion Redes" and "Unidad2_Act2_Jairo". Under "Unidad2_Act2_Jairo", there are four GET requests: B1, B2, B3, and B4. B4 is selected. The main panel shows the details of GET B4, which is a GET request to "https://jsonplaceholder.typicode.com/posts/1/comments". The "Params" tab is active, showing a table with "Key" and "Value" columns. Below the table, the "Body" tab is active, displaying a JSON response. The response is a JSON array of two comment objects. The status bar at the bottom indicates "200 OK" with a response time of 730 ms and a size of 1.52 KB.

```
1 [
2   {
3     "postId": 1,
4     "id": 1,
5     "name": "id labore ex et quam laborum",
6     "email": "eliseo@gardner.biz",
7     "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
8   },
9   {
10    "postId": 1,
11    "id": 2,
12    "name": "quo vero reiciendis velit similique earum",
13    "email": "Jayne_Kuhic@sydney.com",
14    "body": "est natus enim nihil est dolore omnis voluptatem numquam\net omnis occaecati quod ullam at\nvoluptatem error expedita pariatur\nnnihil sint nostrum voluptatem reiciendis et"
15  },
16 ]
```

B5

The screenshot shows the Postman interface with the same workspace. On the left, the sidebar lists "COLLECTIONS" including "Programacion Redes" and "Unidad2_Act2_Jairo". Under "Unidad2_Act2_Jairo", there are nine GET requests (B1-B9) and one POST request, B5. B5 is selected. The main panel shows the details of POST B5, which is a POST request to "https://jsonplaceholder.typicode.com/posts". The "Body" tab is active, displaying a JSON request body. The status bar at the bottom indicates "201 Created" with a response time of 271 ms and a size of 1.07 KB.

```
1 { "title": "naruto", "body": "Anime", "userId": "100" }
```

B6

The screenshot shows the Postman interface for a PUT request. The URL is `https://jsonplaceholder.typicode.com/posts/1`. The request body is a JSON object:

```
1 {
2   "id": 1,
3   "title": "Publicación actualizada",
4   "body": "Contenido actualizado mediante PUT",
5   "userId": 1
6 }
```

The response is a 200 OK status with a response time of 435 ms and a body size of 996 B. The response body is a JSON object:

```
1 {
2   "id": 1,
3   "title": "Publicación actualizada",
4   "body": "Contenido actualizado mediante PUT",
5   "userId": 1
6 }
```

B7

The screenshot shows the Postman interface for a PATCH request. The URL is `https://jsonplaceholder.typicode.com/posts/1`. The request body is a JSON object:

```
1 {
2   "title": "Nuevo título"
3 }
```

The response is a 200 OK status with a response time of 108 ms and a body size of 1.04 KB. The response body is a JSON object:

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "Nuevo título",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
6 }
```

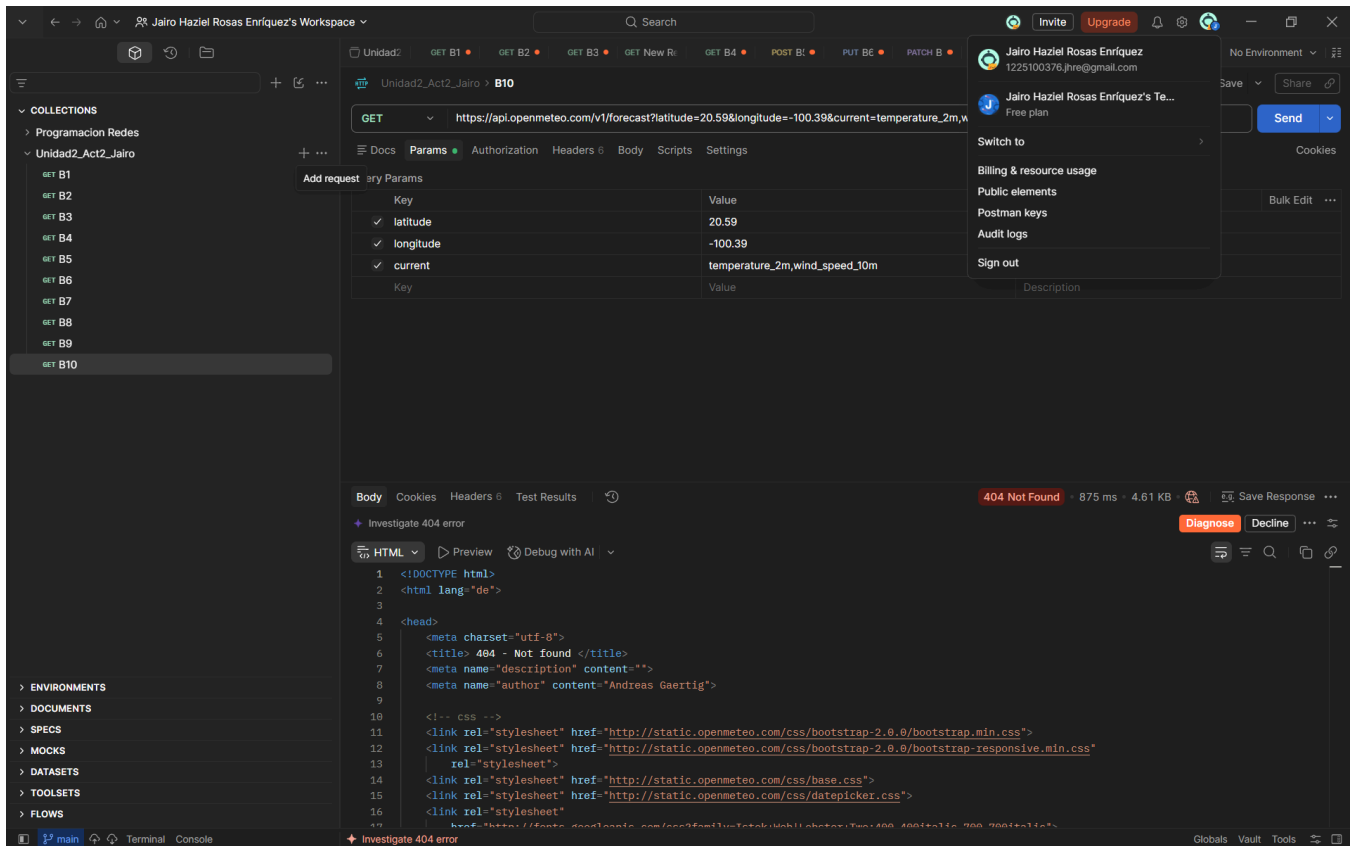
B8

The screenshot shows the Postman application interface. On the left, a sidebar lists collections under 'Jairo Haziel Rosas Enriquez's Workspace'. The main panel displays a DELETE request to 'https://jsonplaceholder.typicode.com/posts/1'. The 'Params' tab is active, showing a table with 'Key' and 'Value' columns. The response section at the bottom shows a '200 OK' status with a response time of 384 ms and a body of '1 {}'. A user menu is open in the top right corner, showing options like 'Switch to', 'Billing & resource usage', 'Public elements', 'Postman keys', 'Audit logs', and 'Sign out'.

B9

The screenshot shows the Postman application interface. On the left, a sidebar lists collections under 'Jairo Haziel Rosas Enriquez's Workspace'. The main panel displays a GET request to 'https://jsonplaceholder.typicode.com/posts/9999'. The 'Params' tab is active, showing a table with 'Key' and 'Value' columns. The response section at the bottom shows a '404 Not Found' status with a response time of 96 ms and a body of '1 {}'. A user menu is open in the top right corner, showing options like 'Switch to', 'Billing & resource usage', 'Public elements', 'Postman keys', 'Audit logs', and 'Sign out'.

B10



Preguntas de la Parte B

1. ¿Qué diferencia observaste entre PUT (B6) y PATCH (B7) en lo que se envía? PUT envía el recurso completo. Reemplaza totalmente el objeto existente en el servidor, PATCH envía solo los campos modificados.
2. ¿Qué encabezado **Content-Type** **regresa el servidor y qué indica?** El encabezado Content-Type indica el formato del cuerpo de la respuesta enviado por el servidor.
3. **En B10, ¿qué parte de la URL es la ruta y cuáles son los parámetros de consulta (query params)?** ¿Qué ruta va antes de un ?

5. Parte C. Consumo de la API con Python

Crea una carpeta `actividad_rest` y un archivo `cliente_rest.py` por cada ejercicio

C1. Primer GET

```
import requests

BASE = "https://jsonplaceholder.typicode.com"
respuesta = requests.get(f"{BASE}/posts/1", timeout=10)

print("Código de estado:", respuesta.status_code)
print("Tipo de contenido:", respuesta.headers["Content-Type"])

publicacion = respuesta.json() # convierte el JSON en un diccionario de Python
print("Título:", publicacion["title"])
```

```
C:\Windows\System32\cmd.e x + v
Microsoft Windows [Versión 10.0.26300.9457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\hazie\OneDrive\Documentos\Programacion de redes\actividad_rest>python C1.py
Jairo Haziel Rosas Enriquez
Código de estado: 200
Tipo de contenido: application/json; charset=utf-8
Título: sunt aut facere repellat provident occaecati excepturi optio reprehenderit

C:\Users\hazie\OneDrive\Documentos\Programacion de redes\actividad_rest>|
```

C2. Recorrer una lista y usar parámetros

```
respuesta = requests.get(f"{BASE}/posts", params={"userId": 3}, timeout=10)
publicaciones = respuesta.json() # lista de diccionarios

print("Total:", len(publicaciones))
for p in publicaciones:
    print(f"{p['id']}>3} | {p['title']}")
```

```
C:\Users\hazie\OneDrive\Documentos\Programacion de redes\actividad_rest>python C2.py
Jairo Haziel Rosas Enriquez
Total: 10
21 | asperiores ea ipsam voluptatibus modi minima quia sint
22 | dolor sint quo a velit explicabo quia nam
23 | maxime id vitae nihil numquam
24 | autem hic labore sunt dolores incidunt
25 | rem alias distinctio quo quis
26 | est et quae odit qui non
27 | quasi id et eos tenetur aut quo autem
28 | delectus ullam et corporis nulla voluptas sequi
29 | iusto eius quod necessitatibus culpa ea
30 | a quo magni similique perferendis

C:\Users\hazie\OneDrive\Documentos\Programacion de redes\actividad_rest>|
```

C3. POST, PUT y DELETE

```
nuevo = {"title": "Prueba de red", "body": "Contenido de ejemplo", "userId": 1}

r = requests.post(f"{BASE}/posts", json=nuevo, timeout=10)
print("POST:", r.status_code, r.json())

r = requests.put(f"{BASE}/posts/1", json=**nuevo, "id": 1, timeout=10)
print("PUT:", r.status_code, r.json())

r = requests.delete(f"{BASE}/posts/1", timeout=10)
print("DELETE:", r.status_code)
```

```
C:\Users\hazie\OneDrive\Documentos\Programacion de redes\actividad_rest>python C3.py
Jairo Haziel Rosas Enriquez
POST: 201 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 101}
PUT: 200 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 1}
DELETE: 200
```

C4. Manejo de errores

Modifica C1 para que consulte `/posts/9999` y maneje el error correctamente:

```

try:
    r = requests.get(f"{BASE}/posts/9999", timeout=10)
    r.raise_for_status() # lanza excepción si el código es 4xx o 5xx
    print(r.json())
except requests.exceptions.HTTPError as e:
    print("Error HTTP:", e)
except requests.exceptions.RequestException as e:
    print("Error de conexión:", e)

```

```

C:\Users\hazie\OneDrive\Documentos\Programacion de redes\actividad_rest>python C4.py
Jairo Haziél Rosas Enriquez
Error HTTP: 404 Client Error: Not Found for url: https://jsonplaceholder.typicode.com/posts/9999

C:\Users\hazie\OneDrive\Documentos\Programacion de redes\actividad_rest>|

```

C5. Consumir otra API (clima)

Escribe una función `obtener_clima(latitud, longitud)` que consulte Open-Meteo y regrese un diccionario con la temperatura y la velocidad del viento. Usa `params` en lugar de armar la URL a mano.

```

def obtener_clima(latitud, longitud):
    url = "https://api.open-meteo.com/v1/forecast"
    parametros = {
        "latitude": latitud,
        "longitude": longitud,
        "current": "temperature_2m,wind_speed_10m",
    }
    r = requests.get(url, params=parametros, timeout=10)
    r.raise_for_status()
    return r.json()["current"]

```

Probarla con tres ciudades distintas (por ejemplo Querétaro: 20.59, -100.39) y mostrar los resultados en una tabla impresa en consola.

```

C:\Users\hazie\OneDrive\Documentos\Programacion de redes\actividad_rest>python C5.py
Jairo Haziél Rosas Enriquez

```

Ciudad	Temperatura	Viento
Queretaro	24.5	14.6
Ciudad de Mexico	21.7	6.0
Guadalajara	25.6	8.4

```

C:\Users\hazie\OneDrive\Documentos\Programacion de redes\actividad_rest>|

```